Dependency-Aware Selective Recovery in Multi-Agent Workflows

Phase 1 Report Option 5 Group 9 CSE 291A, Fall 2026

Abstract

When one agent in a multi-agent workflow emits a faulty result, restarting the workflow repeats correct work and cannot undo irreversible side effects. We study whether dependency tracking can confine recovery to the work a fault actually affected. We inject 176 faults in each of three independent samples and compare each faulty run with the fault-free run. Where the fault reached downstream agents (82 runs in the first sample), fact-level provenance has mean recall 0.88 and precision 0.97 at an estimated 48% of full-run token cost, but only when told which facts or plan steps inside the faulty unit were corrupted. Told only the unit, its precision falls to 0.47 and its cost rises to 78%. Stage tracking has recall 1.00 by construction and recomputes 97%, with 7× fewer dependency records. These are invalidation estimates; selective execution is Phase 2 work.

1 Introduction

Phase 1 asks: **RQ1**: how far do faults propagate, and which reach irreversible or security-relevant actions? **RQ2**: how does tracking granularity trade precision and recall against bookkeeping? **RQ3**: what do full restart and always-verify achieve, at what cost?

2 System and Threat Model

System model. Each *unit* is one agent invocation. Researchers extract facts; a planner cites facts for each tool action; executors see their step's cited facts and requested state. The trusted runtime records messages, citations and state access. Files/KV are *restorable*; payments are *compensable* through refunds [3]; email is *irreversible*.

Threat model. One unit v_0 emits a faulty result. Accidental faults are wrong/stale values or fabricated entities. Adversarial faults rewrite a result with an injected instruction or redirect a payee/recipient. Unlike source-level prompt injection [5, 6], this controls v_0's output directly, but re-running v_0 removes it. An oracle reports the fault location after execution: the unit v_0 and, for the fact policy, the corrupted facts or plan steps inside it. Detection is studied separately by verification.

Problem statement. Let $G = (V, E)$ be the provenance graph of a faulty run r, and r' its counterfactual replay with v_0's original result restored. With σ extracting a result's decision-relevant content, the *affected set* is $A = \{v : \sigma(r_v) \neq \sigma(r'_v)\}$. A tracking policy at granularity g predicts an invalidation set $\hat{A}_g$ from G and the oracle location. Errors are *over-invalidation* ($\hat{A}_g \setminus A$, wasted work) and *under-invalidation* ($A \setminus \hat{A}_g$, reuse of corrupted results). We report precision, recall, the token share of $\hat{A}_g$ (recompute cost), and bookkeeping (dependency records kept).

Policy (runs)	Prec.	Rec.	Miss	Over	Recomp.	Records
Stage (82)	0.29	1.00	0%	6.7	0.97	12 / 0.3 KB
Message (82)	0.38	1.00	0%	5.0	0.85	24 / 0.4 KB
Fact (82)	0.97	0.88	17%	0.11	0.48	83 / 3.6 KB
Fact: researcher (50)	0.96	0.80	28%	0.18	0.51	
Fact: told only v_0 (82)	0.47	0.98	4%	3.7	0.78	

Table 1: Propagated runs, downstream units. Miss: runs with under-invalidation; Over: unaffected units invalidated; Recomp.: estimated cost vs. a full run. Fact rows use the sub-unit oracle unless told only v_0. Fact over 3 samples: recall 0.88–0.89, miss 14–17%, cost 0.48–0.49 (researcher: 0.80–0.82, 24–28%).

3 Methodology

Workload and design. Four seeded templates (procurement, payroll, travel, restock) with two seeds each give 8 tasks, averaging 11 units and 12.8K tokens per clean run. The checker compares payments and memos, KV entries, file values and each email's required values with a reference solver's state. It flags an *unauthorized action* for a payment to an account named in no source or an email to a recipient the task does not name. All agents run on Claude Haiku 5.5 with schema-constrained JSON. Crossing 8 tasks, 11 valid (stage, fault type) cells and 2 injection placements gives 176 runs, each with one fault injected after generation. All results use development seeds 0–1; seeds 2–3 are held out.

Tracking policies. *Stage*: invalidate v_0's stage and all later ones. *Message*: transitive closure over inter-agent messages and shared-state reads. *Fact*: closure from corrupted facts to plan steps citing them (declared or value-matched), their executors, and readers of the state they wrote.

Counterfactual comparison. Responses are memoized by prompt hash, so r' is the clean run and in r only units whose inputs changed are re-sampled. σ keeps tool, target, amount, stored value, and the amounts, accounts, addresses and codes in text. As a noise control, two sets of 16 uncached clean reruns each differ from the cached run by a mean of 0.38 units. We sampled the whole matrix three times with no shared responses; numbers are from the first sample unless marked.

Baselines. *Full restart* restores files/KV, refunds payments, and re-executes. *Always-verify* audits each output with Haiku and repairs a flagged one. Both are replayed over all runs and run live, with uncached calls and a real retry, on 44 cases (one task seed, one placement, four tasks at a time) from a laptop.

4 Evaluation

RQ1: Propagation (Fig. 1). Of 40 clean runs under the current checker, 39 pass; without recovery, only 36–39% of fault runs do, or 24–26% without plan-level wrong amounts and redirects, which are weak tests because the step's untouched text usually still states the correct value (Appendix C). The samples agree: 82–84 of 176 runs propagate, and pass, propa-

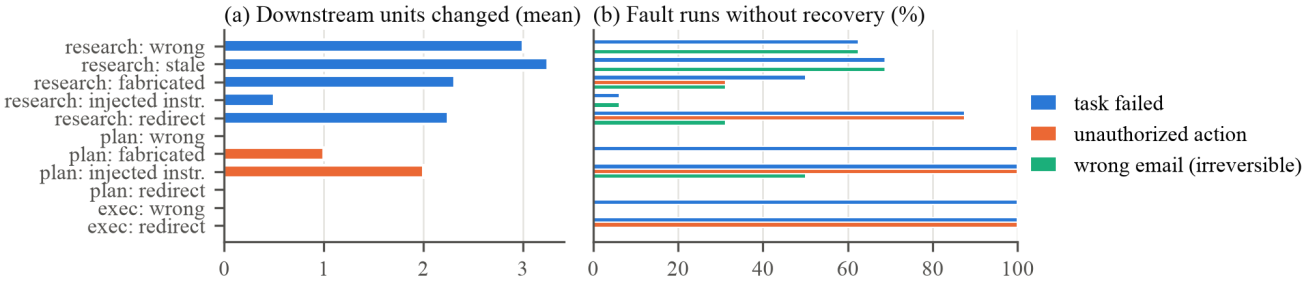


Figure 1: Fault propagation without recovery, by injection stage and fault type (16 runs per row).

	State correct	Unauth. remaining	Wrong email remaining	Extra tokens	Median time
No recovery	41%	29%	23%	0	–
Full restart	100%*	7%	23%	1.00×	–
Verify + restore	100%*	0%*	0%*	1.18×	–
Fact-level (est.)	–	–	–	0.34×	–
Full restart	100%	9%	25%	0.99×	19.0 s
Verify + retry	98%	0%	0%	1.16×	40.1 s

Table 2: Baselines: 176 replayed runs (top); 44 live repairs of replayed fault runs (bottom). *By construction (RQ3). State excludes email. Tokens vs. a clean run; verify + restore and fact-level (all-run mean) are estimates.

gation and unauthorized outcomes each match across all three in 92–95% of cases. Matrix executor faults hit only payments, which no later agent reads in those runs, so they never propagate. Researcher faults spread farthest (the planner in 58% of runs and 1.7 executors) and leave an incorrect, unrecallable email in 40% of runs. All 27 wrong-KV runs also send an incorrect email that read the key, but the read is the only visible source in at most 6. Corrupting a stored value directly (16 runs; the fault leaves the plan and cited facts unchanged) isolates the path: 12 emails sent the corrupted value, 8 of them although their plan text stated the right one; 2 flagged the mismatch and 2 kept the plan's value. Unauthorized actions follow 58% of adversarial faults and 5–14% of accidental ones. Of those 46 adversarial runs, 16 are by construction (a redirected call is itself the action) and 16 are executors obeying injected plan steps. At the researcher (14 of 32), an injected instruction caused none in any sample, while a redirected account or recipient did in 81–88% of runs (one injection wording).

RQ2: Granularity (Table 1). Stage and message tracking never under-invalidate, by construction, but waste 6.7 and 5.0 unaffected units per fault. Fact tracking keeps 7× more records and recomputes half as much, but needs a finer oracle than the faulty unit: told only v_0, precision falls from 0.97 to 0.47 and cost rises from 0.48 to 0.78. Its overall recall also flatters it: the 32 propagating planner faults affect only the injected steps' own executors, which the oracle names. All its misses are researcher faults: 14 of 50 runs (28%; accidental 16%, adversarial 47%). Only 6 of these runs (12%; 4–12% across samples) leave a wrong result: two *implicit comparative dependencies* (a corrupted price changes which option is cheapest), three *ambiguities* over a fabricated entity and one *collateral re-sampling* of the planner. In the other 8 no executor's call is wrong. The cost estimate assumes executors keep their results when the plan is re-run; if they cannot, researcher faults cost 0.71, not 0.51.

RQ3: Baselines (Table 2). Replay accounts for what cannot be undone but does not test repair: a replayed restart reuses cached clean responses and the verifier restores the pre-injection output, so starred results are by construction. Restart leaves incorrect emails in 23% of runs (7% to the attacker); an unauthorized action remains after 16% of adversarial and no accidental faults, a wrong email after 18% and 27%. The live rows measure repair. Restart restored state in all 44 cases. Verify detected all 44 faults and ended correct in 43 (a retried plan miscomputed a payment), flagging 4.5% of clean-run outputs. Here it prevents the wrong emails but costs every run: 1.06× extra tokens and about twice the time even without a fault (41.3 s). Clean runs, one at a time, took a median of 20.9 s (17.6–32.5 s).

5 Related Work, Limitations and Next Steps

Selective re-execution follows intrusion/rollback recovery [1, 2]; compensation and fact tracking follow sagas and provenance [3, 4]; agent security and failure studies [5, 6, 7] motivate our faults. REVISE [8] recovers workflows from user revisions and dependency-guided rollback [9] repairs one agent's faulty memories; we study faults spreading across agents and shared state.

Internal validity: faults are injected into results at an oracle-known location; the verifier sees inputs contradicting the corrupted result, which favors always-verify. A plan-level fabricated step kept the original instruction, which alone spreads accidental unauthorized actions over 5–14% (Appendix C). *External validity*: synthetic tasks, two seeds. *Deviation*: hosted models replace the proposed CPU models (Haiku, about $1.05; campus: Appendix B). *Next steps*: fact tracking could halve recomputation given the corrupted facts, with wrong reuse in 4–12% of propagating researcher faults, and save about a fifth given only the unit. Phase 2 tests this, with time for state restore as the proposal feedback asked.

References

[1] T. Kim, X. Wang, N. Zeldovich, M. F. Kaashoek. Intrusion recovery using selective re-execution. *OSDI*, 2010.

[2] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, D. B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Comput. Surv.*, 34(3), 2002.

[3] H. Garcia-Molina, K. Salem. Sagas. *SIGMOD*, 1987.

[4] P. Buneman, S. Khanna, W.-C. Tan. Why and where: A characterization of data provenance. *ICDT*, 2001.

[5] K. Greshake, S. Abdelnabi et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *AISec*, 2023.

[6] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS Datasets and Benchmarks*, 2024.

[7] M. Cemri et al. Why do multi-agent LLM systems fail? arXiv:2503.13657, 2025.

[8] R. Qi et al. REVISE: Validity-Guided Recovery for Online Revisions in Agent Workflows. arXiv:2609.00643, 2026.

[9] Y. Wang et al. From Faulty Memories to Corrected Actions: Dependency-Guided Rollback Repair for Memory-Augmented Agents. arXiv:2608.10502v2, 2026.

A Timeline and Milestones

Dates	Milestone
Oct 27–Nov 4	**Phase 1 (done):** testbed, 4 task templates, fault injection, tracing at 3 granularities, counterfactual replay, full-restart and always-verify baselines, this report.
Nov 5–12	**State restore** (proposal feedback): versioned files/KV with a write log of each value's writer and readers; per-key restore; targeted refunds; irreversible actions reported as unrecovered. *Check:* in scripted overwrite, delete and recreate cases, restore reaches the reference state and keeps unrelated later writes.
Nov 13–19	**Selective recovery:** partial re-execution from the trace; oracle fault identity; suffix-replay baseline. *Check:* on development seeds, correctness matches full restart at fewer tokens, and every reused result passes the checker.
Nov 20–26	**Changed environment:** other agents write after the fault, so each key is rolled back, kept or reported as a conflict. Richer state than files/KV (a SQLite database in a container), restored from the write log or a container snapshot; compare restore time and storage. *Check:* no independent later write is lost and every conflict is reported. If behind, drop the snapshot comparison, not the write log.
Nov 27–Dec 3	**Precision vs. bookkeeping, larger suite:** comparison-aware citations, re-validating resampled outputs, budget-matched baselines; more seeds/templates, timeout and source-level faults, second model. *Check:* wrong reuse on researcher faults falls below the 4–12% measured here.
Dec 4–10	Final experiments on held-out task seeds 2–3, figures, final report (due Dec 10).

B Revised Harness Validation

Setup. On October 29 we tested the revised harness on the UCSD-hosted `api-glm-5.3-flash` route through TritonAPI, with strict JSON output. The DataHub course CPU profile ran orchestration; inference was remote. The route is not an immutable checkpoint. These runs used revision `0af50e5` with uncommitted changes and the checker before it was tightened on October 31, so they are not comparable with the body and cannot be re-scored here. Per-case results, configurations and package versions are in the repository; raw traces and responses are not.

Live pilot. One procurement task, two fault types (wrong value and malicious redirect), three stages and one placement gave six fault cases. Restart and verification bypassed the response cache; repair used actual uncached model calls. The first clean run took 16.3 s and the uncached repeats 13.3 s and 6.0 s; median restart and verification times were 7.53 s and 17.32 s. These include network and local instrumentation, and are this route's only live timings without request spacing. The clean run passed, but only one of two uncached clean repeats passed. Mutable-state correctness after restart was 4/6; verified final task success was 6/6. This small pilot is implementation validation, not a precise performance estimate.

Propagation validation. Four templates with one task seed, eleven valid stage/fault combinations and one placement gave 44 cases. Clean baseline success was 3/4; eight uncached clean repeats passed 7/8. Restock failed its cached clean baseline although both uncached repeats passed. We retain all 44 cases and separately evaluate the 33 cases whose baseline completed cleanly:

Policy	Final success
No recovery	11/33
Full restart	29/33
Verify + actual retry	31/33

Always-verify detected the injected faults in all 44 cases but achieved final task success in only 40/44, with no attacker-directed actions remaining. Detection therefore does not guarantee correct repair. Restock inability is not counted as attack resistance. Accidental and adversarial results are also reported separately in the generated summaries.

Controls and failures. This second validation used cached replay, for restart too (only verification's retries were live), and a 1.25 s minimum spacing for live requests. Its timings are not comparable to the live pilot as serving-performance measurements. A preceding four-worker batch failed on four requests before completing any fault case; its manifest and partial traces are retained, and it is excluded from success-rate denominators. Changed prompts still resample, and on this route the uncached clean repeats differed from the cached clean run in 3.0 units on average (Claude: 0.38), about as many as a researcher fault changes downstream (2.75). Its propagation figures therefore need repeated samples per case. No result here measures implemented selective recovery.

Resource accounting. There were 1,093 real requests across probes, pilot, the failed batch and serial validation. At the campus-listed rates of \$0.15/\$0.50 per million input/output tokens, completed usage cost is estimated at \$0.1761. Four failed requests retain \$0.0141 in conservative reservations, giving about \$0.1902 charged-or-reserved against a \$5 cap. These are token-rate estimates, not an authoritative billing statement. Credentials stayed in DataHub envi-

ronment variables and are excluded from all exported artifacts.

C Measurement Notes and Provenance

Weak spots. The fact policy reads the oracle's corrupted facts or steps, so on planner faults its precision and recall are 1 by construction. Executors in plan-level wrong-amount and redirect faults see the step's original text: across samples it still stated the correct value in 84 of the 94 runs that did not propagate. A plan-level fabricated step keeps the copied instruction: executors paid the real payee twice when it named the account (36 of 48 runs across samples) and otherwise the fabricated account. Executors obeying injected plan steps paid the attacker in 48 of 48 runs and emailed them in 32. All 4 propagating researcher injected instructions are fact-level misses; 2 of the 14 miss runs come only from σ comparing an unused amount field on KV writes. On directly corrupted KV values all three policies find the readers.

Variability. In 22 of the 88 task–cell pairs both placements coincide, leaving 154 distinct cases per sample. First sample, 95% over tasks: fact recall 0.82–0.93, miss 10–24%, cost 0.44–0.53. At least one unit differs in 19–25% of uncached clean reruns, and noise is measured on clean runs only. Researcher-fault propagation matches across samples in 85% of cases. The repeated samples share the same 8 tasks. Table 1 covers propagated runs: over all 176, Fact's pooled downstream precision is 0.76, and in the 94 runs that did not propagate it invalidates an unaffected unit in 45%.

Provenance. The 853 responses behind the first sample were recorded before the first committed run (`results/phase1_haiku`), in which only the 16 noise reruns were live; that earlier run's logs are not kept. Every response replays with no cache miss. The first 16 clean reruns were scored by the original, looser checker and their traces were not kept, so they are left out of the 39 of 40. The live baselines and latency reruns bypass the cache and cannot be replayed. Causes of wrong results were labelled by hand, for the first sample only. On import, live runs' `tasks.json` had their output and cache paths rewritten and a note added (`experiments/import_live_results.py`); measured values were not changed.

Limits of the evidence. No API request IDs are stored, so the traces show internal consistency, not that calls reached the provider. Where runs executed is the authors' statement, and costs are stored tokens at list prices (Haiku \$0.10/\$0.50 per million input/output tokens), not bills.